

Lectura 9

Resolución numérica de Sistemas de Ecuaciones No Lineales

Dr. Luciano Ponzellini Marinelli
lucianoponzellini@uca.edu.ar

Facultad de Química e Ingeniería
Rosario, Argentina

Notas de clase
14 octubre 2025





Organización

INTRODUCCIÓN

MÉTODO DE PUNTO FIJO

CONVERGENCIA DE PF

MÉTODO DE NEWTON-RAPHSON PARA SENL

CONSIDERACIONES



INTRODUCCIÓN

Introducción

- Muchos problemas en ciencia e ingeniería se modelan matemáticamente como un Sistema de Ecuaciones No Lineales (SENL)

$$\mathbf{F}(\mathbf{x}) = \mathbf{0}$$

de n ecuaciones (no lineales) y n incógnitas donde:

$$\mathbf{x} = (x_1, x_2, \dots, x_n) \in \Omega \subset \mathbb{R}^n$$

y

$\mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_n(\mathbf{x}))$ vector de funciones

- Existen métodos numéricos para hallar raíces del SENL es

Método de Punto Fijo (PF)
Método de Newton-Raphson

EL MÉTODO DE PUNTO FIJO

Ejemplo introductorio

- Consideremos los campos escalares definidos sobre $\mathbb{R}^2$:

$$f_1(x, y) = x^2 - 2x - y + 0.5$$

$$f_2(x, y) = x^2 + 4y^2 - 4$$

- Sea el SENL:

$$\begin{cases} f_1(x, y) = 0 \\ f_2(x, y) = 0 \end{cases}$$

Estas ecuaciones definen implícitamente curvas en el plano $z = 0$

- Una raíz es un punto $(p^*, q^*) \in \mathbb{R}^2$ donde se cortan las curvas:

$$\begin{cases} f_1(p^*, q^*) = 0 \\ f_2(p^*, q^*) = 0 \end{cases}$$

¡Queremos un método para hallar estas raíces!

Interpretación geométrica

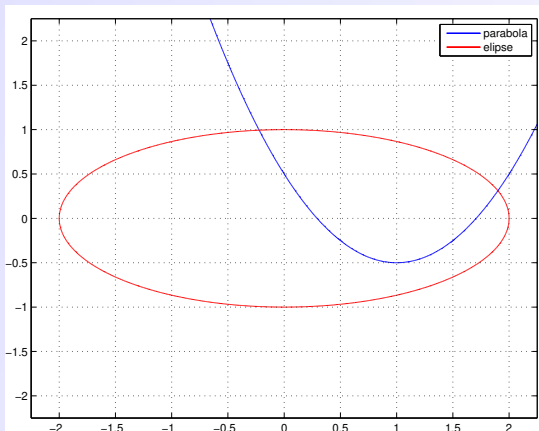


Figura: Intersección de una elipse y una parábola

Ver `t_contour.m/.sce`.

Ejemplo introductorio: Punto Fijo

- Despejando algebraicamente x e y de alguna forma particular:

$$f_1(x, y) = 0 \Leftrightarrow x = \frac{x^2 - y + 0.5}{2}$$

$$f_2(x, y) = 0 \Leftrightarrow y = \frac{-x^2 - 4y^2 + 8y + 4}{8}$$

- Definimos las funciones **generatrices**:

$$g_1(x, y) := \frac{x^2 - y + 0.5}{2}$$

$$g_2(x, y) := \frac{-x^2 - 4y^2 + 8y + 4}{8}$$

Ejemplo introductorio: Punto Fijo

- Generamos la sucesión $\{(p_k, q_k)\}_{k \geq 0}$

$$p_{k+1} = g_1(p_k, q_k) = \frac{p_k^2 - q_k + 0.5}{2} \quad k \geq 0$$

$$q_{k+1} = g_2(p_k, q_k) = \frac{-p_k^2 - 4q_k^2 + 8q_k + 4}{8} \quad k \geq 0$$

tomando un valor de aproximación inicial (p_0, q_0)

- Hay otras posibles generatrices:

$$\tilde{g}_1(x, y) := \frac{-x^2 + 4x + y - 0.5}{2}$$

$$\tilde{g}_2(x, y) := \frac{-x^2 - 4y^2 + 11y + 4}{11}$$

¿Por qué son generatrices?

Ejemplo introductorio: Generatrices

- Primero tenemos que definir las generatrices como *m-file*:

```
function Y = G1(X)
% Función generatriz para del SENL Punto Fijo
% ENTRADA: X = [ X(1) X(2) ] vector fila de dos componentes
% SALIDA: Y = [ Y(1) Y(2) ] vector fila de dos componentes

Y(1) = ( X(1)^2 - X(2) + 0.5) / 2;
Y(2) = (-X(1)^2 - 4*X(2)^2 + 8*X(2) + 4) / 8;

end
```

Esta función define las generatrices que luego pasaremos como parámetro a PuntoFijo.m como cadena de caracteres (string)

Implementación *PuntoFijo.m*

```
function [Pk,iter] = PuntoFijo(Gnom,P0,tol,itmax)
% Datos
% - Gnom: (string) función m-file que definen las generatrices
% - P0: estimación inicial
% - tol: tolerancia
% - itmax: número iteraciones máximas
% Resultado
% - Pk: aproximación solución
% - iter: número de iteraciones realizadas

fprintf('Método iterativo de Punto Fijo \ n \ n')
fprintf('Iter      Pk      Qk \ n')
iter = 0;
fprintf(' %3.0f%15.7f%15.7f \ n', iter, P0)
...
```

Implementación *PuntoFijo.m*

```

while 1
    Pk = feval(Gnom,P0);
    err = abs(norm(Pk-P0));
    relerr = err / (norm(Pk)+eps);
    if (err < tol || relerr < tol)
        disp('Se alcanzó la tolerancia.')
        disp('Resultado final: ')
        fprintf('%15.7f%15.7f \ n',Pk), break
    end
    if iter > itmax
        disp('Número de iteraciones máximo sobrepasado. '), break
    end
    iter = iter + 1;
    P0 = Pk;
    fprintf('%3.0f%15.7f%15.7f \ n', iter, Pk)
end

```

Ejemplo introductorio: *PuntoFijo.m*

```
>> [Pk,iter] = PuntoFijo('G1',[0 1],1e-9,15)
```

Metodo iterativo de Punto Fijo

Iter	Pk	Qk
0	0.0000000	1.0000000
1	-0.2500000	1.0000000
2	-0.2187500	0.9921875
3	-0.2221680	0.9939880
4	-0.2223147	0.9938121
5	-0.2221941	0.9938029
6	-0.2222163	0.9938095
7	-0.2222147	0.9938083
8	-0.2222145	0.9938084
9	-0.2222146	0.9938084
10	-0.2222146	0.9938084

¡CONVERGE!

Se alcanzó la tolerancia.

Nota: En **Scilab** no hace falta pasar el argumento G1 entre apóstrofes ('')

Ejemplo introductorio: *PuntoFijo.m*

```
>> [Pk,iter] = PuntoFijo('G1',[2 0],1e-9,15)
```

Metodo iterativo de Punto Fijo

Iter	Pk	Qk
0	2.0000000	0.0000000
1	2.2500000	0.0000000
2	2.7812500	-0.1328125
3	4.1840820	-0.6085510
4	9.3075467	-2.4820360
5	44.8062311	-15.8910907
6	1011.9947186	-392.6042650
7	512263.2073904	-205477.8225378
:	:	
13	Inf	-Inf
14	Inf	-Inf
15	Inf	-Inf

¡DIVERGE!

Nota: En **Scilab** no hace falta pasar el argumento G1 entre apóstrofes ('')

Ejemplo introductorio: otras generatrices

- Consideremos las generatrices $\tilde{g}_1$ y $\tilde{g}_2$ dadas anteriormente:

$$f_1(x, y) = 0 \Leftrightarrow x = \tilde{g}_1(x, y) = \frac{-x^2 + 4x + y - 0.5}{2}$$

$$f_2(x, y) = 0 \Leftrightarrow y = \tilde{g}_2(x, y) = \frac{-x^2 - 4y^2 + 11y + 4}{11}$$

- Consideremos la sucesión obtenida a partir de estas:

$$p_{k+1} = \tilde{g}_1(p_k, q_k) = \frac{-p_k^2 + 4p_k + q_k - 0.5}{2} \quad k \geq 0$$

$$q_{k+1} = \tilde{g}_2(p_k, q_k) = \frac{-p_k^2 - 4q_k^2 + 11q_k + 4}{11} \quad k \geq 0$$

tomando alguna aproximación inicial (p_0, q_0)

Ejemplo introductorio: otras generatrices

- Ahora definimos las nuevas generatrices como otro *m-file*:

```
function Y = G2(X)
% Función generatriz para del SENL Punto Fijo
% ENTRADA: X = [ X(1) X(2) ] vector fila de dos componentes
% SALIDA: Y = [ Y(1) Y(2) ] vector fila de dos componentes

Y(1) = ( -X(1)^2 + 4*X(1) + X(2) - 0.5) / 2;
Y(2) = (-X(1)^2 - 4*X(2)^2 + 11*X(2) + 4) / 11;

end
```

Esta función define las nuevas generatrices que también pasaremos como parámetro a PuntoFijo.m como cadena de caracteres (string)

Ejemplo introductorio: otras generatrices

```
>> [Pk,iter] = PuntoFijo('G2',[2 0],1e-9,40)
```

Metodo iterativo de Punto Fijo

Iter	Pk	Qk
0	2.0000000	0.0000000
1	1.7500000	0.0000000
2	1.7187500	0.0852273
3	1.7530629	0.1776676
⋮	⋮	⋮
10	1.9023137	0.3123981
⋮	⋮	⋮
20	1.9006772	0.3112196
⋮	⋮	⋮
30	1.9006767	0.3112186

¡CONVERGE!

Se satisface la tolerancia.

Nota: En **Scilab** no hace falta pasar el argumento G2 entre apóstrofes ('')

Definición y esquema de punto fijo en 2 dimensiones

- Un **punto fijo** de un sistema de ecuaciones bidimensional

$$\begin{cases} x = g_1(x, y) \\ y = g_2(x, y) \end{cases}$$

es un punto $(p^*, q^*) \in \Omega \subset \mathbb{R}^2$ del dominio de g_1 y g_2 donde

$$\begin{aligned} p^* &= g_1(p^*, q^*) \\ q^* &= g_2(p^*, q^*) \end{aligned}$$

siendo g_1 y g_2 campos escalares y $(x, y) \in \mathbb{R}^2$.

- Para las funciones del sistema de ecuaciones bidimensional, el **esquema de iteración de punto fijo** es:

$$\begin{cases} p_{k+1} = g_1(p_k, q_k) & k \geq 0 \\ q_{k+1} = g_2(p_k, q_k) & k \geq 0 \end{cases}$$

para cierto (p_0, q_0) dado inicialmente.

Definición y esquema de punto fijo en 3 dimensiones

- Un **punto fijo** de un sistema de ecuaciones tridimensional

$$\begin{cases} x = g_1(x, y, z) \\ y = g_2(x, y, z) \\ z = g_3(x, y, z) \end{cases}$$

es un punto $(p^*, q^*, r^*) \in \Omega \subset \mathbb{R}^3$ del dominio de g_1, g_2 y g_3 donde

$$p^* = g_1(p^*, q^*, r^*)$$

$$q^* = g_2(p^*, q^*, r^*)$$

$$r^* = g_3(p^*, q^*, r^*)$$

siendo g_1, g_2 y g_3 campos escalares y $(x, y, z) \in \mathbb{R}^3$.

- Para las funciones del sistema de ecuaciones tridimensional, el **esquema de iteración de punto fijo** es:

$$\begin{cases} p_{k+1} = g_1(p_k, q_k, r_k) & k \geq 0 \\ q_{k+1} = g_2(p_k, q_k, r_k) & k \geq 0 \\ r_{k+1} = g_3(p_k, q_k, r_k) & k \geq 0 \end{cases}$$

para cierto (p_0, q_0, r_0) dado inicialmente.

Definición y esquema de punto fijo en n-dimensiones

- Análogamente, un **punto fijo** del sistema n-dimensional

$$\mathbf{x} = \mathbf{G}(\mathbf{x})$$

es un punto $\mathbf{x}^* = (x_1^*, \dots, x_n^*) \in \Omega \subset \mathbb{R}^n$ del dominio de $\mathbf{G}$ donde

$$\mathbf{x}^* = \mathbf{G}(\mathbf{x}^*)$$

siendo $\mathbf{G} = (g_1, \dots, g_n)$ vector de funciones y $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$.

- Análogamente, para el sistema de ecuaciones n-dimensional, el **esquema de iteración de punto fijo** es:

$$\mathbf{x}_{k+1} = \mathbf{G}(\mathbf{x}_k) \quad k \geq 0$$

siendo $\mathbf{G} = (g_1, \dots, g_n)$, $\mathbf{x}_k = (x_1^{(k)}, \dots, x_n^{(k)})$, $k \geq 1$ y

$\mathbf{x}_0 = (x_1^{(0)}, \dots, x_n^{(0)})$ dado inicialmente.

CONVERGENCIA DE PUNTO FIJO

Convergencia 'cerca' de puntos fijos en $\mathbb{R}^2$

Teorema. Caso bidimensional

Sean las generatrices g_1 y g_2 con derivadas parciales continuas en un dominio que contiene un punto fijo en (p^*, q^*) .

Sea (p_0, q_0) 'suficientemente cerca' de (p^*, q^*) y se satisface:

$$\left| \frac{\partial g_1(x, y)}{\partial x} \right| + \left| \frac{\partial g_1(x, y)}{\partial y} \right| < 1$$

$$\left| \frac{\partial g_2(x, y)}{\partial x} \right| + \left| \frac{\partial g_2(x, y)}{\partial y} \right| < 1$$

para todo punto (x, y) en un *entorno* de (p^*, q^*) .

Entonces la iteración de punto fijo genera una sucesión $\{(p_k, q_k)\}_{k \geq 0}$ convergente al punto fijo (p^*, q^*) .

Si las condiciones de acotación no se cumplen entonces la iteración de Punto Fijo podría diverger o bien converger

Convergencia 'cerca' de puntos fijos en $\mathbb{R}^3$

Teorema. Caso tridimensional

Sean las generatrices g_1, g_2 y g_3 con derivadas parciales continuas en un dominio que contiene un punto fijo en (p^*, q^*, r^*) .

Sea (p_0, q_0, r_0) 'suficientemente cerca' de (p^*, q^*, r^*) y se satisface:

$$\left| \frac{\partial g_1(x, y, z)}{\partial x} \right| + \left| \frac{\partial g_1(x, y, z)}{\partial y} \right| + \left| \frac{\partial g_1(x, y, z)}{\partial z} \right| < 1$$

$$\left| \frac{\partial g_2(x, y, z)}{\partial x} \right| + \left| \frac{\partial g_2(x, y, z)}{\partial y} \right| + \left| \frac{\partial g_2(x, y, z)}{\partial z} \right| < 1$$

$$\left| \frac{\partial g_3(x, y, z)}{\partial x} \right| + \left| \frac{\partial g_3(x, y, z)}{\partial y} \right| + \left| \frac{\partial g_3(x, y, z)}{\partial z} \right| < 1$$

para todo punto (x, y, z) en un entorno de (p^*, q^*, r^*) .

Entonces la iteración de punto fijo genera una sucesión $\{(p_k, q_k, r_k)\}_{k \geq 0}$ convergente al punto fijo (p^*, q^*, r^*) .

Si las condiciones de acotación no se cumplen entonces la iteración de Punto Fijo podría diverger o bien converger

La matriz jacobiana

Definición.

- Sean $f_1(x, y)$ y $f_2(x, y)$ con derivadas parciales continuas y sea $\mathbf{F} = (f_1, f_2)$. Definimos en cada (x, y) la **matriz jacobiana** $\mathbf{J}_{\mathbf{F}}(x, y)$:

$$\mathbf{J}_{\mathbf{F}}(x, y) = \begin{bmatrix} \nabla f_1(x, y) \\ \nabla f_2(x, y) \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1(x, y)}{\partial x} & \frac{\partial f_1(x, y)}{\partial y} \\ \frac{\partial f_2(x, y)}{\partial x} & \frac{\partial f_2(x, y)}{\partial y} \end{bmatrix}$$

- Sean $f_1(x, y, z)$, $f_2(x, y, z)$ y $f_3(x, y, z)$ con derivadas parciales continuas. Definimos en cada (x, y, z) la **matriz jacobiana** $\mathbf{J}_{\mathbf{F}}(x, y, z)$ como:

$$\mathbf{J}_{\mathbf{F}}(x, y, z) = \begin{bmatrix} \nabla f_1(x, y, z) \\ \nabla f_2(x, y, z) \\ \nabla f_3(x, y, z) \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1(x, y, z)}{\partial x} & \frac{\partial f_1(x, y, z)}{\partial y} & \frac{\partial f_1(x, y, z)}{\partial z} \\ \frac{\partial f_2(x, y, z)}{\partial x} & \frac{\partial f_2(x, y, z)}{\partial y} & \frac{\partial f_2(x, y, z)}{\partial z} \\ \frac{\partial f_3(x, y, z)}{\partial x} & \frac{\partial f_3(x, y, z)}{\partial y} & \frac{\partial f_3(x, y, z)}{\partial z} \end{bmatrix}$$

Ejemplos de la matriz jacobiana

EJEMPLO 1:

Sean g_1 y g_2 las generatrices con derivadas parciales continuas:

$$g_1(x, y) = \frac{x^2 - y + 0.5}{2} \quad \text{y} \quad g_2(x, y) = \frac{-x^2 - 4y^2 + 8y + 4}{8}$$

La matriz jacobiana $\mathbf{J}_F(x, y)$ resulta:

$$\mathbf{J}_F(x, y) = \begin{bmatrix} \frac{\partial g_1(x, y)}{\partial x} & \frac{\partial g_1(x, y)}{\partial y} \\ \frac{\partial g_2(x, y)}{\partial x} & \frac{\partial g_2(x, y)}{\partial y} \end{bmatrix} = \begin{bmatrix} x & -\frac{1}{2} \\ -\frac{x}{4} & -y + 1 \end{bmatrix}$$

EJEMPLO 2:

Sean $\tilde{g}_1$ y $\tilde{g}_2$ las nuevas generatrices con derivadas parciales continuas:

$$\tilde{g}_1(x, y) = \frac{-x^2 + 4x + y - 0.5}{2} \quad \text{y} \quad \tilde{g}_2(x, y) = \frac{-x^2 - 4y^2 + 11y + 4}{11}$$

¿Cómo queda la matriz jacobiana $\mathbf{J}_F(x, y)$?

La matriz jacobiana generalizada

Definición.

- Análogamente, si $f_1, \dots, f_m \in C^1(\Omega)$ con derivadas parciales continuas y si $\mathbf{F} = (f_1, \dots, f_m)$, definimos en cada $\mathbf{x} = (x_1, \dots, x_n)$ la **matriz jacobiana** $\mathbf{J}_{\mathbf{F}}(\mathbf{x})$ como:

$$\mathbf{J}_{\mathbf{F}}(x_1, \dots, x_n) = \begin{bmatrix} \nabla f_1(\mathbf{x}) \\ \nabla f_2(\mathbf{x}) \\ \vdots \\ \nabla f_m(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \frac{\partial f_1(\mathbf{x})}{\partial x_2} & \cdots & \frac{\partial f_1(\mathbf{x})}{\partial x_n} \\ \frac{\partial f_2(\mathbf{x})}{\partial x_1} & \frac{\partial f_2(\mathbf{x})}{\partial x_2} & \cdots & \frac{\partial f_2(\mathbf{x})}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m(\mathbf{x})}{\partial x_1} & \frac{\partial f_m(\mathbf{x})}{\partial x_2} & \cdots & \frac{\partial f_m(\mathbf{x})}{\partial x_n} \end{bmatrix}$$

- Notemos que la matriz tiene m filas y n columnas (orden $m \times n$).
- Suele usarse indistintamente $\mathbf{J}_{\mathbf{F}}(\mathbf{x})$ o $\mathbf{J}(\mathbf{x})$ sin subíndice.

Convergencia 'cerca' de puntos fijos en $\mathbb{R}^n$

Teorema. Caso n-dimensional

Sean las funciones $\mathbf{G} = (g_1, \dots, g_n)$ con derivadas parciales continuas en un dominio que contiene un punto fijo $\mathbf{x}^*$.

Sea $\mathbf{x}_0$ *suficientemente cerca* de $\mathbf{x}^*$, y se satisface que:

$$\|\mathbf{J}(\mathbf{x}^*)\|_{\infty} < 1$$

para todo punto (x, y) en un *entorno* de $\mathbf{x}^*$.

Entonces la iteración de punto fijo

$$\mathbf{x}_{k+1} = \mathbf{G}(\mathbf{x}_k), \quad k \geq 1$$

genera una sucesión $\{\mathbf{x}_k\}_{k \geq 0}$ convergente al punto fijo $\mathbf{x}^*$.

Observación: En el caso bidimensional y tridimensional pedimos que la norma infinito de la matriz jacobiana $\|\mathbf{J}\|_{\infty} < 1$ en un entorno del punto fijo

Ejemplo introductorio: región de convergencia

- Con las primeras generatrices de 'G1.m':

$$\left| \frac{\partial g_1}{\partial x} \right| + \left| \frac{\partial g_1}{\partial y} \right| = |x| + \left| \frac{-1}{2} \right| < 1 \quad (1)$$

$$\left| \frac{\partial g_2}{\partial x} \right| + \left| \frac{\partial g_2}{\partial y} \right| = \left| \frac{-x}{4} \right| + |-y + 1| < 1 \quad (2)$$

- Analíticamente podemos hallar una **región de convergencia**.

De (1) obtenemos

$$|x| < 0.5$$

De (2) obtenemos

$$\frac{|x|}{4} + |y - 1| < 1$$

- Inicializando en $(x_0, y_0) = (0, 1)$ tenemos garantizada la convergencia al punto fijo aproximado

$$(x^*, y^*) \approx (-0.2222146, 0.9938084)$$

- Pero inicializando en $(x_0, y_0) = (2, 0)$ no podemos garantizar la convergencia (en nuestro caso diverge)

Ejemplo introductorio: región de convergencia

- Con las segundas generatrices generatrices de 'G2.m':

$$\left| \frac{\partial \tilde{g}_1}{\partial x} \right| + \left| \frac{\partial \tilde{g}_1}{\partial y} \right| = |-x + 2| + \left| \frac{1}{2} \right| < 1 \quad (3)$$

$$\left| \frac{\partial \tilde{g}_2}{\partial x} \right| + \left| \frac{\partial \tilde{g}_2}{\partial y} \right| = \left| \frac{-2x}{11} \right| + \left| \frac{-8}{11}y + 1 \right| < 1 \quad (4)$$

- Analíticamente podemos hallar otra **región de convergencia**.

De (3) obtenemos:

$$|x - 2| < 0.5$$

De (4) obtenemos:

$$\frac{|x|}{4} + \left| y - \frac{11}{8} \right| < \frac{11}{8}$$

- Inicializando en $(x_0, y_0) = (2, 0)$ *no* podemos garantizar la convergencia, pero aún así converge al punto fijo aproximado

$$(x^*, y^*) \approx (1.9006767, 0.3112186)$$

MÉTODO DE NEWTON-RAPHSON PARA SENL

Desarrollo NR caso bidimensional

- Vamos a construir el método de NR en el caso bidimensional:

$$\begin{cases} f_1(x, y) = 0 \\ f_2(x, y) = 0 \end{cases}$$

- Consideremos la transformación:

$$\begin{cases} u = f_1(x, y), \\ v = f_2(x, y), \end{cases} \quad (5)$$

como una transformación del plano xy en el plano uv .

- Estamos interesados en el comportamiento de la transformación cerca del punto (x_0, y_0) cuya imagen es el punto (u_0, v_0)

$$\begin{cases} u_0 = f_1(x_0, y_0), \\ v_0 = f_2(x_0, y_0), \end{cases} \quad (6)$$

asumiendo que f_1 y f_2 tienen derivadas parciales continuas.

Desarrollo NR caso bidimensional

- La serie de Taylor en dos dimensiones para cada campo escalar es:

$$\begin{cases} f_1(x, y) = f_1(x_0, y_0) + \frac{\partial f_1(x_0, y_0)}{\partial x}(x - x_0) + \frac{\partial f_1(x_0, y_0)}{\partial y}(y - y_0) + \text{términos sup.} \\ f_2(x, y) = f_2(x_0, y_0) + \frac{\partial f_2(x_0, y_0)}{\partial x}(x - x_0) + \frac{\partial f_2(x_0, y_0)}{\partial y}(y - y_0) + \text{términos sup.} \end{cases}$$

- Utilizando solo los términos lineales de Taylor:

$$\begin{cases} f_1(x, y) \approx f_1(x_0, y_0) + \frac{\partial f_1(x_0, y_0)}{\partial x}(x - x_0) + \frac{\partial f_1(x_0, y_0)}{\partial y}(y - y_0), \\ f_2(x, y) \approx f_2(x_0, y_0) + \frac{\partial f_2(x_0, y_0)}{\partial x}(x - x_0) + \frac{\partial f_2(x_0, y_0)}{\partial y}(y - y_0). \end{cases}$$

- Usando las ecuaciones (5) y (6) resulta

$$\begin{cases} u - u_0 \approx \frac{\partial f_1(x_0, y_0)}{\partial x}(x - x_0) + \frac{\partial f_1(x_0, y_0)}{\partial y}(y - y_0), \\ v - v_0 \approx \frac{\partial f_2(x_0, y_0)}{\partial x}(x - x_0) + \frac{\partial f_2(x_0, y_0)}{\partial y}(y - y_0). \end{cases}$$

Desarrollo NR caso bidimensional

- Si usamos la **matriz Jacobiana** $\mathbf{J}(x, y)$

$$\mathbf{J}(x, y) = \begin{bmatrix} \frac{\partial f_1(x, y)}{\partial x} & \frac{\partial f_1(x, y)}{\partial y} \\ \frac{\partial f_2(x, y)}{\partial x} & \frac{\partial f_2(x, y)}{\partial y} \end{bmatrix}$$

- En forma matricial resulta

$$\begin{bmatrix} u - u_0 \\ v - v_0 \end{bmatrix} \approx \begin{bmatrix} \frac{\partial f_1(x_0, y_0)}{\partial x} & \frac{\partial f_1(x_0, y_0)}{\partial y} \\ \frac{\partial f_2(x_0, y_0)}{\partial x} & \frac{\partial f_2(x_0, y_0)}{\partial y} \end{bmatrix} \begin{bmatrix} x - x_0 \\ y - y_0 \end{bmatrix}.$$

- En forma compacta

$$\begin{bmatrix} u - u_0 \\ v - v_0 \end{bmatrix} \approx \mathbf{J}(x_0, y_0) \begin{bmatrix} x - x_0 \\ y - y_0 \end{bmatrix}.$$

Desarrollo NR caso bidimensional

- Llamando:

$$\begin{cases} \Delta u = u - u_0, \\ \Delta v = v - v_0, \end{cases} \quad \begin{cases} \Delta x = x - x_0, \\ \Delta y = y - y_0, \end{cases}$$

obtenemos

$$\begin{bmatrix} \Delta u \\ \Delta v \end{bmatrix} \approx \mathbf{J}(x_0, y_0) \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}. \quad (7)$$

- Supongamos que (x^*, y^*) es una raíz del SENL original, es decir:

$$\begin{cases} u = f_1(x^*, y^*) = 0, \\ v = f_2(x^*, y^*) = 0, \end{cases}$$

entonces resulta

$$\begin{cases} \Delta u = u - u_0 = f_1(x^*, y^*) - f_1(x_0, y_0) = 0 - f_1(x_0, y_0) = -f_1(x_0, y_0), \\ \Delta v = v - v_0 = f_2(x^*, y^*) - f_2(x_0, y_0) = 0 - f_2(x_0, y_0) = -f_2(x_0, y_0), \end{cases}$$

Desarrollo NR caso bidimensional

- Entonces la Ec. (7) queda:

$$\mathbf{J}(x_0, y_0) \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \approx \begin{bmatrix} -f_1(x_0, y_0) \\ -f_2(x_0, y_0) \end{bmatrix} \quad (8)$$

- Llamando

$$\Delta \mathbf{P} = \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}, \quad \mathbf{F}(x_0, y_0) = \begin{bmatrix} f_1(x_0, y_0) \\ f_2(x_0, y_0) \end{bmatrix}$$

obtenemos la Ec. (8) en forma compacta

$$\mathbf{J}(x_0, y_0) \Delta \mathbf{P} \approx -\mathbf{F}(x_0, y_0) \quad (9)$$

- Si la matriz jacobiana $\mathbf{J}(x_0, y_0)$ es invertible, podemos despejar $\Delta \mathbf{P}$:

$$\Delta \mathbf{P} \approx -\mathbf{J}^{-1}(x_0, y_0) \mathbf{F}(x_0, y_0) \quad (10)$$

siendo $\mathbf{J}^{-1}(x_0, y_0)$ la inversa de la matriz jacobiana en (x_0, y_0) .

Desarrollo NR caso bidimensional

- Esto nos proporciona la siguiente aproximación:

$$\Delta \mathbf{P} = \mathbf{P}_1 - \mathbf{P}_0$$

puesto que usando la Ec. (10) resulta

$$\mathbf{P}_1 = \mathbf{P}_0 + \Delta \mathbf{P} \approx \mathbf{P}_0 - \mathbf{J}^{-1}(\mathbf{P}_0)\mathbf{F}(\mathbf{P}_0)$$

es decir

$$\mathbf{P}_1 \approx \mathbf{P}_0 - \mathbf{J}^{-1}(\mathbf{P}_0)\mathbf{F}(\mathbf{P}_0) \quad (11)$$

- Podemos construir el esquema iterativo de la siguiente manera:

$$\begin{aligned} \Delta \mathbf{P} &\approx -\mathbf{J}^{-1}(\mathbf{P}_k)\mathbf{F}(\mathbf{P}_k), \quad k \geq 0 \\ \mathbf{P}_{k+1} &= \mathbf{P}_k + \Delta \mathbf{P}, \quad k \geq 0. \end{aligned} \quad (12)$$

inicializando en una aproximación inicial $\mathbf{P}_0$.

Evitamos el cálculo costoso computacional de la inversa $\mathbf{J}^{-1}(\mathbf{P}_k)$ resolviendo el sistema de ecuaciones lineales (9)

Ejemplo introductorio (mismo PF)

- Consideremos el sistema no lineal:

$$\begin{cases} x^2 - 2x - y + 0,5 = 0 & (\text{parábola}) \\ x^2 + 4y^2 - 4 = 0 & (\text{elipse}) \end{cases}$$

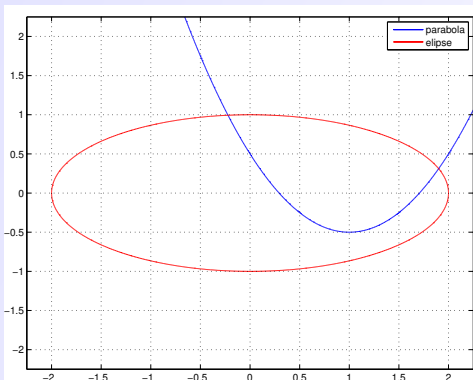


Figura: Intersección de la elipse y la parábola

Ejemplo introductorio

- Consideremos los campos escalares:

$$\begin{cases} f_1(x, y) = x^2 - 2x - y + 0,5 \\ f_2(x, y) = x^2 + 4y^2 - 4 \end{cases}$$

Usaremos NR tomando como estimación inicial $\mathbf{P}_0 = \begin{bmatrix} 2.00 \\ 0.25 \end{bmatrix}$

- La función vectorial y la matriz jacobiana son:

$$\mathbf{F}(x, y) = \begin{bmatrix} x^2 - 2x - y + 0,5 \\ x^2 + 4y^2 - 4 \end{bmatrix} \quad \mathbf{J}(x, y) = \begin{bmatrix} 2x - 2 & -1 \\ 2x & 8y \end{bmatrix}$$

- Evaluadas en la estimación inicial

$$\mathbf{F}(2.00, 0.25) = \begin{bmatrix} 0.25 \\ 0.25 \end{bmatrix} \quad \mathbf{J}(2.00, 0.25) = \begin{bmatrix} 2.00 & -1.00 \\ 4.00 & 2.00 \end{bmatrix}$$

Ejemplo introductorio

- Resolviendo el sistema lineal 2×2 con eliminación gaussiana

$$\begin{bmatrix} 2.00 & -1.00 \\ 4.00 & 2.00 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = - \begin{bmatrix} 0.25 \\ 0.25 \end{bmatrix}$$

obtenemos como solución

$$\begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} -0.09375 \\ 0.06250 \end{bmatrix}$$

- La siguiente aproximación es

$$\mathbf{P}_1 = \mathbf{P}_0 + \Delta \mathbf{P} = \begin{bmatrix} 2.00 \\ 0.25 \end{bmatrix} + \begin{bmatrix} -0.09375 \\ 0.06250 \end{bmatrix} = \begin{bmatrix} 1.90625 \\ 0.31250 \end{bmatrix}$$

- Análogamente obtenemos

$$\mathbf{P}_2 = \begin{bmatrix} 1.900691 \\ 0.311213 \end{bmatrix}, \quad \mathbf{P}_3 = \begin{bmatrix} 1.900677 \\ 0.311219 \end{bmatrix}$$

Esquema del método de Newton-Raphson

Supongamos que hemos obtenido la aproximación $\mathbf{P}_k = (x_k, y_k)$:

- **Paso 1.** Evaluamos la función vectorial

$$\mathbf{F}(\mathbf{P}_k) = \begin{bmatrix} f_1(x_k, y_k) \\ f_2(x_k, y_k) \end{bmatrix}$$

- **Paso 2.** Evaluamos la matriz jacobiana

$$\mathbf{J}(\mathbf{P}_k) = \begin{bmatrix} \frac{\partial f_1(x_k, y_k)}{\partial x} & \frac{\partial f_1(x_k, y_k)}{\partial y} \\ \frac{\partial f_2(x_k, y_k)}{\partial x} & \frac{\partial f_2(x_k, y_k)}{\partial y} \end{bmatrix}$$

- **Paso 3.** Calculamos $\Delta \mathbf{P}$ resolviendo el sistema lineal

$$\mathbf{J}(\mathbf{P}_k) \Delta \mathbf{P} = -\mathbf{F}(\mathbf{P}_k)$$

- **Paso 4.** Calculamos la siguiente aproximación

$$\mathbf{P}_{k+1} = \mathbf{P}_k + \Delta \mathbf{P}$$

- **Paso 5.** Si $\|\mathbf{P}_{k+1} - \mathbf{P}_k\| < tol$ o $\frac{\|\mathbf{P}_{k+1} - \mathbf{P}_k\|}{\|\mathbf{P}_{k+1}\| + eps} < tol$ detengo la iteración. Sino actualizo y sigo.

Ejemplo introductorio: la función vectorial

- Primero tenemos que definir la función vectorial como *m-file*:

```
function Y = F1(X)
% Función vectorial ejemplo introductorio Newton-Raphson SENL (ver
lecturas)
% ENTRADA: X = [ X(1) X(2) ] vector columna de 2x1
% SALIDA: Y = [ Y(1) Y(2) ] vector columna de 2x1
%
% Luciano Ponzellini Marinelli (2022)
Y(1) = X(1)^2 - 2*X(1) - X(2) + 0.5;
Y(2) = X(1)^2 + 4*X(2)^2 - 4.0;
Y=Y';
end
```

Esta función define la función vectorial del sistema no lineal que luego pasaremos a NewtonRaphsonSENL.m como argumento

Ejemplo introductorio: la matriz jacobiana

- Segundo tenemos que derivar para definir la matriz jacobiana:

```
function J = JF1(X)
% Matriz Jacobiana ejemplo introductorio Newton-Raphson SENL (ver
lecturas)
% ENTRADA: X = [ X(1) X(2) ] vector columna de 2x1
% SALIDA: J matriz 2x2
%
% Luciano Ponzellini Marinelli (2022)
J(1,1) = 2*X(1)-2.0;
J(1,2) = -1.0;
J(2,1) = 2*X(1);
J(2,2) = 8*X(2);
end
```

Esta función define la matriz jacobiana que pasaremos como argumento

Implementación *NewtonRaphsonSENL.m*

```
function [Pk,iter,err] = NewtonRaphsonSENL(Fnom,Jnom,P0,tol,itmax)
% Entrada:
% - Fnom funcion vectorial que define el sistema de ecuaciones no lineal
% - Jnom funcion que define la matriz jacobiana del sistema
% - P0 estimación inicial vector columna 2x1
% - tol tolerancia
% - itmax número iteraciones máximas
% Salida:
% - Pk solución aproximada
% - iter número de iteraciones realizadas
% - err estimación del error de Pk
% Usa:
% - utiliza Gauss.m para resolver el sistema lineal
%
% Luciano Ponzellini Marinelli (2022)
...
```

Implementación *NewtonRaphsonSENL.m*

```

...
Pk = P0;
iter= 1;
while 1
    FX = feval(Fnom,Pk);
    Jacob = feval(Jnom,Pk);
    dP = Gauss(Jacob,-FX);
    err = norm(dP);
    relerr = err / (norm(Pk) + eps);
    Pk = Pk + dP;
    fprintf(' %3.0f %12.6f %12.6f',iter,Pk)
    if err < tol || relerr < tol
        disp(' Se satisface la tolerancia. ')
        disp(' Resultado final: '), fprintf(' %12.6f \n',Pk), return
    end
    iter = iter + 1;
    if iter > itmax
        disp(' Número de iteraciones máximo alcanzado. '),
        disp(' Resultado parcial: '), fprintf(' % 12.6f ',Pk), break
    end
end
end

```

Ejemplo introductorio: *NewtonRaphsonSENL.m*

```
>> [Pk,iter,err] = NewtonRaphsonSENL('F1','JF1',[2.00 0.25]',1e-9,15);
```

Método de Newton-Raphson bidimensional

Iter	Pk	Qk
0	2.000000	0.250000
1	1.906250	0.312500
2	1.900691	0.311213
3	1.900677	0.311219
4	1.900677	0.311219

¡CONVERGE!

Se satisface la tolerancia.

Resultado final:

1.900677

0.311219

```
>> iter
```

```
iter = 4
```

```
>> err
```

```
err = 9.8962e-11
```

Nota: En **Scilab** no hace falta pasar F1 y JF1 entre apóstrofes (')

Ejemplo introductorio: *NewtonRaphsonSENL.m*

```
>> [Pk,iter,err] = NewtonRaphsonSENL('F1','JF1',[0 1]',1e-9,15);
```

Método de Newton-Raphson bidimensional

Iter	Pk	Qk
0	0.000000	1.000000
1	-0.250000	1.000000
2	-0.222561	0.993902
3	-0.222215	0.993808
4	-0.222215	0.993808
5	-0.222215	0.993808

¡CONVERGE!

Se satisface la tolerancia.

Resultado final:

-0.222215

0.993808

```
>> iter
```

```
iter = 5
```

```
>> err
```

```
err = 1.5215e-15
```

CONSIDERACIONES DE NEWTON-RAPHSON

Extensión natural de NR

- Recordemos que en el caso de una variable, vimos que la **iteración de N-R** es de la forma:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} = x_n - [f'(x_n)]^{-1} f(x_n), \quad n \geq 0$$

siendo x_0 una aproximación inicial y siempre que $f'(x_n) \neq 0$.

- Notemos que la fórmula (12) es la generalización de lo anterior:

$$\mathbf{P}_{k+1} = \mathbf{P}_k - [\mathbf{J}(\mathbf{P}_k)]^{-1} \mathbf{F}(\mathbf{P}_k), \quad k \geq 0.$$

siendo $\mathbf{P}_0$ una estimación inicial y siempre que la matriz jacobiana $\mathbf{J}(\mathbf{P}_k)$ sea invertible, es decir, exista su inversa $[\mathbf{J}(\mathbf{P}_k)]^{-1}$.

NR como caso particular de PF (1D)

- Notemos que en el caso de una variable la **iteración de N-R**:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

puede verse como una iteración de punto fijo.

- Llamamos a la generatriz :

$$g(x) = x - \frac{f(x)}{f'(x)}$$

- Entonces buscar una raíz de una ecuación no lineal $f(x) = 0$, equivale a encontrar un punto fijo x^* de la ecuación anterior:

$$x^* = g(x^*) = x^* - \frac{f(x^*)}{f'(x^*)}$$

NR como caso particular de PF (n -D)

- Notemos que en el caso de n -variables la **iteración de N-R**:

$$\mathbf{P}_{k+1} = \mathbf{P}_k - [\mathbf{J}(\mathbf{P}_k)]^{-1} \mathbf{F}(\mathbf{P}_k), \quad k \geq 0.$$

puede verse como una iteración de punto fijo.

- Llamamos a la generatriz :

$$\mathbf{G}(\mathbf{X}) = \mathbf{X} - [\mathbf{J}_F(\mathbf{X})]^{-1} \mathbf{F}(\mathbf{X}).$$

- Entonces buscar una raíz de un SENL $\mathbf{F}(\mathbf{X}) = \mathbf{0}$, equivale a encontrar un punto fijo $\mathbf{X}^*$ del sistema anterior:

$$\mathbf{X}^* = \mathbf{G}(\mathbf{X}^*) = \mathbf{X}^* - [\mathbf{J}_F(\mathbf{X}^*)]^{-1} \mathbf{F}(\mathbf{X}^*).$$

Bibliografía

- **[Kel2003]** C.T. Kelley, *Solving Nonlinear Equations with Newton's Method*, SIAM, Philadelphia, 2003.
- **[MaFi2000]** J.M. Mathews - K.D. Fink, *Métodos Numéricos con MATLAB*, 3era Ed., Pearson Prentice Hall, Madrid, 2000.
- **[Mole2008]** C.B. Moler, *Numerical computing with MATLAB*, 2da. Ed., SIAM, Philadelphia, 2008.
- **[SPMK2023]** J.A. Semitiel, L. Ponzellini Marinelli & A.I. Kurdobrin, *Notas de Clases de Matemática Aplicada de Ingeniería Eléctrica e Ingeniería Electrónica*, FCEIA, UNR, 2023.
- **[SiSo2010]** J.W. Signorelli, J. Sorribas, *Notas de Clases de Informática Aplicada y Métodos Computacionales*, FCEIA, UNR, 2010.